

WEEK1

Task1

To Install Linux VM, Configure GCC, Setup Git Repository,
Create Project Structure, Understand Shell Architecture,
Build Initial Makefile

Output:

```
root@DESKTOP-VMN932K:~# cd OSSP
root@DESKTOP-VMN932K:~/OSSP# tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 19 files
```

Task2

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing

Source code:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main() {

    pid_t pid;

    printf("Current Process PID : %d\n", getpid());

    printf("Current Process PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0) {

        perror("fork failed");

        return 1;

    }

    if (pid == 0) {

        printf("Child PID : %d\n", getpid());

        printf("Child PPID : %d\n", getppid());

        printf("\nChild will execute 'ls' using exec()\n");
```

```

sleep(2);

execl("/bin/ls", "ls", "-l", NULL);

// This executes only if exec fails
perror("exec failed");
exit(1);
}

else {
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);
    printf("\nParent waiting for child...\n");
    wait(NULL);
    printf("\nChild process has terminated.\n");
    printf("Parent process completed.\n");
}

return 0;
}

```

Output:

```

Current Process PID : 525
Current Process PPID : 304
Parent PID : 525
Child PID : 526

Parent waiting for child...
Child PID : 526
Child PPID : 525

-rw-r--r--  1 root root    45 Aug  7 14:14 source.txt
-rwxr-xr-x  1 root root 16408 Jul 31 15:17 source_code
-rw-r--r--  1 root root   798 Jul 31 15:17 source_code.c
-rw-r--r--  1 root root   8102 Aug  7 14:28 trace.txt

Child process has terminated.
Parent process completed.

```

Task3

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

Source Code:

```
#include <stdio.h>

#include <string.h>

int main() {
    char input[100];

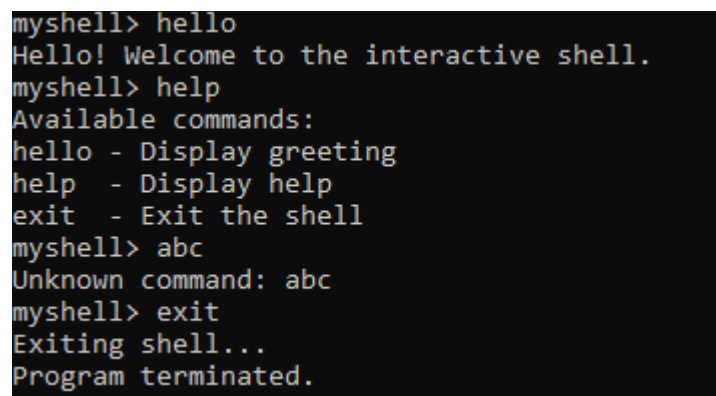
    while (1) {
        printf("myshell> ");
        if (fgetc(input, sizeof(input), stdin) == NULL) {
            printf("\nInput error or EOF detected.\n");
            break;
        }
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0) {
            continue;
        }
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }
        if (strcmp(input, "hello") == 0) {
            printf("Hello! Welcome to the interactive shell.\n");
        }
        else if (strcmp(input, "help") == 0) {
            printf("Available commands:\n");
        }
    }
}
```

```
        printf("hello - Display greeting\n");
        printf("help - Display help\n");
        printf("exit - Exit the shell\n");
    }
    else {
        printf("Unknown command: %s\n", input);
    }
}

printf("Program terminated.\n");

return 0;
}
```

Output:



```
myshell> hello
Hello! Welcome to the interactive shell.
myshell> help
Available commands:
hello - Display greeting
help - Display help
exit - Exit the shell
myshell> abc
Unknown command: abc
myshell> exit
Exiting shell...
Program terminated.
```

Task4

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction

Source Code:

```
#include <stdio.h>

#include <unistd.h>

#include <termios.h>

#include <string.h>

#define BUFFER_SIZE 100

int main() {
    struct termios oldt, newt;
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;
    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);
    printf("Type a command and press Enter.\n");
    printf("Type 'exit' to quit.\n\n");

    printf("> ");

    while (1) {
        read(STDIN_FILENO, &ch, 1);
        if (ch == '\n' || ch == '\r') {
```

```

buffer[index] = '\0';

printf("\nYou entered: %s\n", buffer);
if (strcmp(buffer, "exit") == 0) {
    break;
}
if (strcmp(buffer, "hello") == 0) {
    printf("Hello! Welcome.\n");
}
else if (strcmp(buffer, "help") == 0) {
    printf("Commands: hello, help, exit\n");
}
else if (index == 0) {
    printf("No input entered.\n");
} else {
    printf("Command received successfully.\n");
}
index = 0;
printf("> ");
}
else if (ch == 127 || ch == 8) {

    if (index > 0) {
        index--;
        printf("\b \b");
        fflush(stdout);
    }
}
}

```

```

else if (ch >= 32 && ch <= 126) {

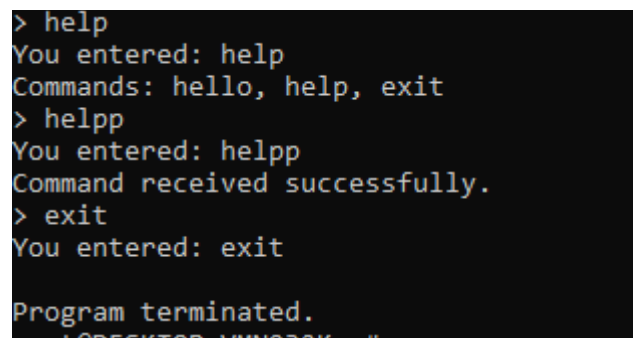
    if (index < BUFFER_SIZE - 1) {
        buffer[index++] = ch;
        printf("%c", ch);
        fflush(stdout);
    }
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);
printf("\nProgram terminated.\n");

return 0;
}

```

Output:



```

> help
You entered: help
Commands: hello, help, exit
> helpp
You entered: helpp
Command received successfully.
> exit
You entered: exit
Program terminated.

```